

College ROI Analyzer

A return-on-investment tool for US college and major choices, built on Department of Education College Scorecard data

Project plan · Prepared June 18, 2026 · Data source last updated June 10, 2026

One-line summary. Combine tuition, debt, graduation rates, and IRS-verified post-graduation earnings from the College Scorecard to compute an honest, program-level return on investment, then present it so a high-school student and their family can compare schools and majors for the actual financial outcome each is likely to produce.

1. Why this project is worth building

The College Scorecard is published by the US Department of Education and is unusually credible because its earnings figures come from real IRS tax records rather than surveys. It covers roughly 6,000 institutions and, critically, breaks earnings and debt down by field of study, so you can measure a specific major at a specific school rather than relying on a school-wide average that hides huge variation. Most public college-comparison sites never use the program-level data, which is exactly where the value sits.

The decision it informs (where to go to college and what to study) is one of the largest financial commitments most families ever make, and they make it with surprisingly little reliable outcome data. That combination of a strong public dataset, a clear analytical question, and a high-stakes real-world decision is what makes this both a solid technical portfolio piece and a project with genuine social value.

2. The data foundation

Layer	What you get	Notes
Institution-level	Tuition and fees, net price by family-income bracket, median federal debt, completion and retention rates, admission rate, demographics	Most complete; few gaps
Field-of-study level	Median earnings and median debt by program (4-digit CIP code) and credential level	The core of real ROI; pooled award years through 2018-19/2019-20
Earnings timing	Median earnings measured ~1 year after completion (and longer horizons at institution level)	Reflects past cohorts, not a forecast

Access. Two routes. The bulk download (full CSV/data files, no rate limits) is best for a reproducible pipeline. The live API returns JSON, requires a free API key, and is rate-limited to 1,000 requests per IP per hour. Field-of-study data is nested as an array under each institution and can be expanded with an `all_programs_nested=true` parameter. For this project the bulk files are the right primary source, with the API reserved for spot lookups.

Plan around this: program-level earnings and debt are *suppressed* (left blank) wherever a cohort is too small to protect privacy. Smaller schools and niche majors will have real gaps. Handle them honestly (show "insufficient data") rather than imputing over them silently.

3. What "ROI" will mean

The tool should support a layered definition, from simple to honest:

- **Net cost vs early earnings** — net price (for a given family income) against median earnings after graduation. Simple, but ignores completion risk.
- **Payback period** — how many years of the earnings premium it takes to recoup the net investment plus debt.
- **Earnings premium vs counterfactual** — earnings compared against a high-school-diploma-only baseline for the same state, so you measure what the degree actually adds.
- **Completion-weighted ROI** — expected earnings weighted by the probability of graduating. A single adjustment that makes the tool smarter than most ranking sites.

4. The two build paths

Both paths share the same data and analysis work. They differ only in how results reach the user. Path A precomputes everything and ships a static site with no backend to host. Path B runs a real Python backend behind a React frontend.

Dimension	Path A — Static precompute	Path B — FastAPI + React
How it works	CI pipeline computes all ROI metrics, writes Parquet / JSON / DuckDB-WASM; browser queries static files	FastAPI serves ROI queries live from DuckDB; React frontend calls the API
Backend hosting	None required	Render free tier or ~\$5/mo always-on
Cold starts	None	Yes on free tier (30–60s wake)
Fit for this data	Excellent — data is static between annual refreshes	Good, but live compute is not strictly needed
Technical growth	Moderate (CI, data engineering, frontend)	Higher (full-stack API design, deployment)
Ongoing cost	\$0	\$0–\$5/mo
Best when	You want the cheapest, most robust public tool	You want full-stack experience as the main goal

Recommendation: start with Path A. It fits the dataset almost perfectly (the data only changes once a year), costs nothing, never cold-starts, and still exercises strong data-engineering and frontend skills. If full-stack backend experience is itself a goal, graduate the same project to Path B later by putting a FastAPI layer in front of the DuckDB you already built. You do not have to choose permanently up front.

5. Phased plan and effort

Effort is given in focused part-time days. "Shared" phases apply to both paths; path-specific work is marked.

Phase	Work	Effort	Applies to
0. Setup	Repo, API key, environment, project structure	0.5 day	Shared
1. Data pipeline	Pull bulk Scorecard files, load to DuckDB/Parquet, document fields, handle suppressed values	2–3 days	Shared
2. ROI engine	Implement the four ROI metrics, completion weighting, state HS-diploma baseline join	2–3 days	Shared
3. Analysis + viz	Notebook: rankings, debt-vs-earnings scatter, ROI by major, public vs private; sanity-check outputs	2–3 days	Shared
4a. Static frontend	Precompute step in CI; build query UI (major + income picker), tables, charts, maps over static files	4–7 days	Path A
4b. API + React	FastAPI endpoints over DuckDB; React frontend; wire up; deploy backend + frontend separately	8–14 days	Path B
5. Predictive model	scikit-learn model to estimate earnings where cohorts are thin; document uncertainty	2–4 days	Optional, both
6. Equity + geo lens	ROI by income bracket; GeoPandas maps of access/outcomes; cost-of-living adjustment	2–4 days	Optional, both
7. Refresh automation	GitHub Actions: annual full refresh + monthly update check	1 day	Shared
8. Polish + docs	README, methodology write-up, data caveats, accessibility pass	1–2 days	Shared

Total effort by scope

Scope	Includes	Path A total	Path B total
Minimum viable	Phases 0–3 + basic frontend + refresh	~1.5–2 weeks part-time	~3–4 weeks part-time
Polished public tool	Add phases 5 or 6 + full polish	~3–4 weeks part-time	~5–7 weeks part-time
Full ambitious build	Everything, both optional phases	~5–6 weeks part-time	~6–9 weeks part-time

6. Cost

Item	Path A	Path B	Note
Scorecard data + API	\$0	\$0	Public; free API key
Compute / dev	\$0	\$0	Local machine
CI (GitHub Actions)	\$0	\$0	Free tier ample for annual/monthly runs

Backend hosting	\$0	\$0–\$5/mo	Render free (cold starts) or paid always-on
Frontend hosting	\$0	\$0	GitHub Pages / Netlify / Cloudflare Pages
Domain (optional)	~\$12/yr	~\$12/yr	Only if you want a custom URL
Effective total	\$0 (~\$12/yr with domain)	\$0–\$5/mo	

2026 note: backend free tiers tightened this year. Fly.io dropped its free tier for new accounts, Railway's free option is now effectively a one-time \$5 trial then \$1/mo for a tiny service, and Render's free web service spins down after 15 minutes idle and cut bandwidth to 5 GB/mo. Static hosting (Path A) sidesteps all of this.

7. Infrastructure

Component	Choice	Why
Storage / query	DuckDB + Parquet	Fast columnar analytics, runs locally, no server; DuckDB-WASM can run in-browser for Path A
Analysis	Python + Pandas	Standard, well-supported
Modeling	scikit-learn	Earnings estimation for thin cohorts
Geospatial	GeoPandas	Optional maps of access and outcomes
Backend (Path B)	FastAPI	Lightweight, modern Python API; good learning step
Frontend	React or vanilla JS	Builds on your Spelling Bee / Web Speech API frontend experience
Automation	GitHub Actions	Annual full refresh + monthly update check
Hosting	GitHub Pages / Netlify / Cloudflare Pages (+ Render for Path B)	Free static hosting; Render only if a live backend is needed

8. Pros and cons

Overall project

Pros: credible public data with IRS-verified earnings; program-level detail that competitors ignore; clear social value; flexible scope from a weekend analysis to a deployed public tool; reusable, well-documented data.

Cons: suppressed/missing data for small cohorts needs careful, honest handling; earnings reflect past cohorts and must be framed as historical, not guaranteed; ROI is one input among many in a college decision, so framing and messaging matter; field-of-study earnings currently sit at a ~1-year horizon, which understates long-run value.

Path A — Static precompute

Pros: zero hosting cost; no cold starts; extremely robust (nothing to crash); simplest to maintain; perfect fit for annually-updated static data.

Cons: no live server-side compute; all query logic runs in the browser; less full-stack/backend experience; large precomputed files need thoughtful structuring.

Path B — FastAPI + React

Pros: meaningful full-stack growth (API design, deployment); flexible live querying; more impressive architecture for a portfolio.

Cons: backend hosting cost or cold-start friction on free tiers; more moving parts to build, deploy, and keep alive; live compute is not actually required for this data.

9. Key risks and how to handle them

Risk	Mitigation
Suppressed program-level data	Show "insufficient data" explicitly; only model where defensible; never silently impute
Earnings are historical	State the cohort years and a clear "past results, not a promise" caveat in the UI
Over-claiming ROI as a verdict	Frame as one important input; surface non-financial context and limitations
Field mismatches across files	Build a documented schema mapping in phase 1; add data-validation tests
Scope creep	Ship the minimum-viable scope first; treat phases 5–6 as optional add-ons

10. Suggested first steps

- Request a free Scorecard API key and download the most recent bulk data files.
- Stand up the repo with a DuckDB load script (phase 1) and confirm you can read both institution and field-of-study tables.
- Implement the simplest ROI metric end to end for one major across a handful of schools, to validate the data joins before scaling.
- Decide Path A vs B only after the analysis phase — by then you will know how much interactivity you actually need.

Sources: College Scorecard data and documentation (collegescorecard.ed.gov), data last updated June 10, 2026; 2026 hosting free-tier status for Render, Railway, and Fly.io.